

# Faculty Timetable Generator Using C

## C Mini Project

### TEAM MEMBERS:

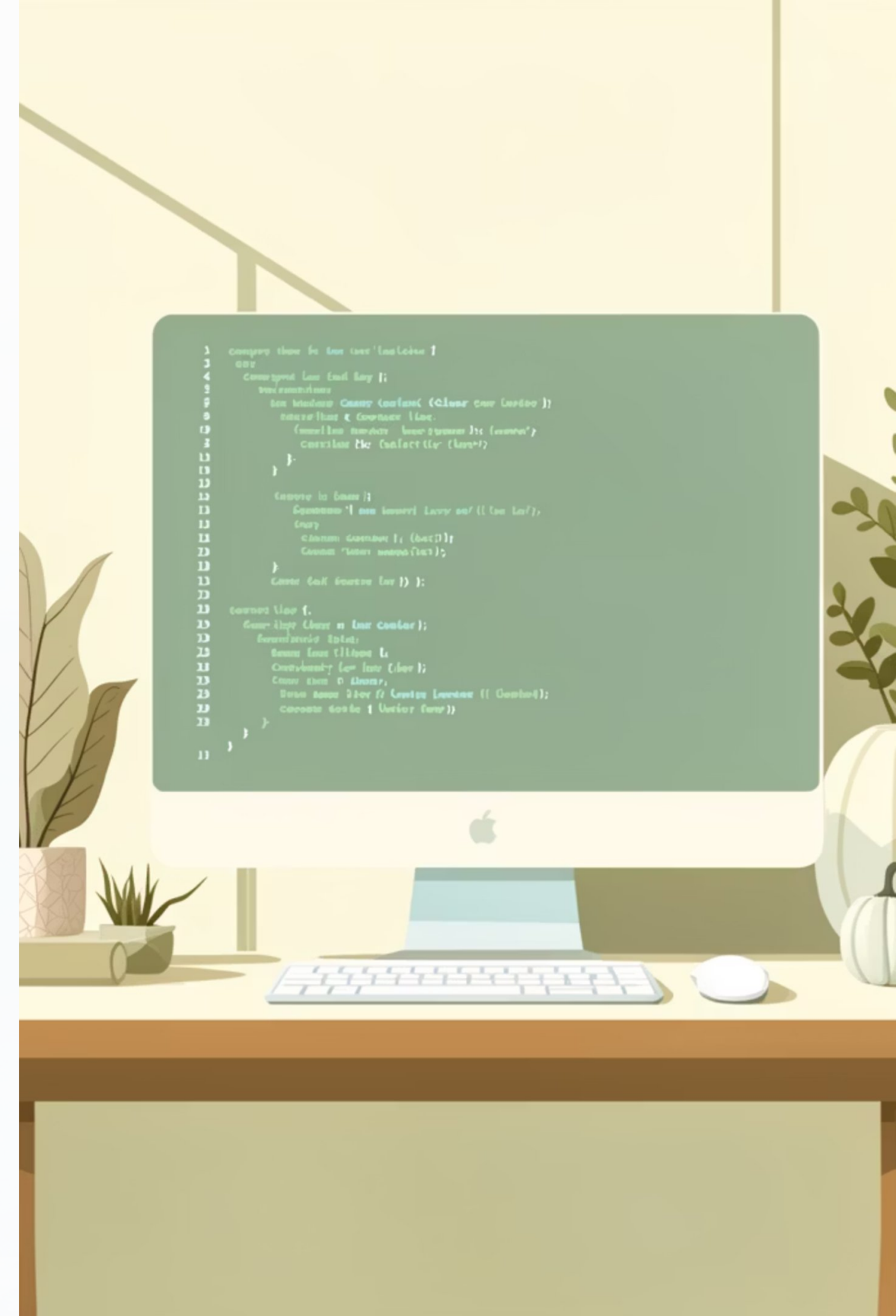
1. Anya Tripathi
2. Tanush Sujith
3. Arjun Shringarpure
4. Diya Palliwal
5. Veer Thakkar

### PRN NUMBER :

1. 1262253395
2. 1262253179
3. 1262251954
4. 1262252573
5. 1262253843

Subject : FOUNDATION OF PROGRAMMING LAB

Teacher : Prof. Nikhil Soniminde





# Introduction

Timetable management is a critical task in educational institutions.



In many colleges, timetable scheduling is:

- Done manually
- Time-consuming
- Prone to human errors
- Difficult to modify

**Common Problems:**

- Faculty assigned to two classes at same time
- Batch assigned multiple subjects in same period
- No centralized data storage

To solve this, we developed a **Faculty Timetable Generator** using **C** language.

# Problem Statement

Educational institutions require an automated system that prevents scheduling conflicts and manages faculty timetables efficiently. Manual scheduling leads to double booking, conflicts, and poor time management.

1

## **Prevents Faculty Clashes**

Ensures no double booking of instructors

2

## **Prevents Batch Clashes**

Ensures no overlapping batch assignments

3

## **Persistent Storage**

Stores timetable data permanently in CSV files

4

## **Real-Time Updates**

Displays slot availability and allows easy updates



# Project Objectives

01

## Develop Timetable System

Build using C programming language with modular approach

03

## Prevent Conflicts

Implement automated conflict-checking mechanism

05

## Separate File Management

Maintain distinct files for faculty and batches

02

## Store Data in CSV

Maintain timetable data in comma-separated value files

04

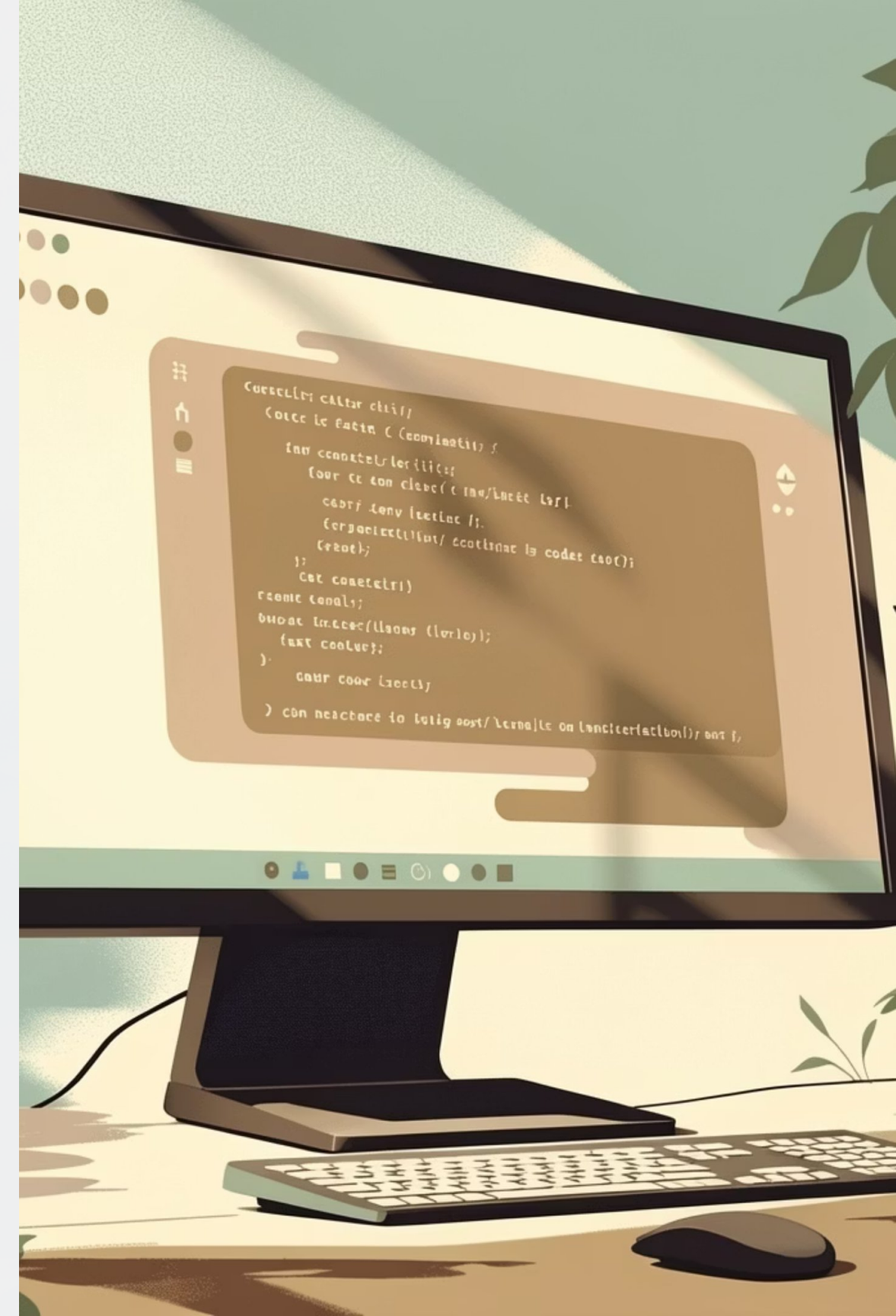
## Slot Status Tracking

Provide FREE, OCCUPIED, and BLOCKED status indicators

06

## Modular Design

Use organized, scalable code structure



# Technologies & Concepts Used

## Development Tools

- GCC Compiler
- Code Editor (VS Code / Dev C++)

## Core Concepts

- Structures
- 2D Arrays
- File Handling (CSV)
- Header Files
- Modular Programming
- String Handling
- Conditional Statements
- Loops

## ❓ Programming Language

- C Programming

# System Architecture

The project is divided into multiple modules for improved readability, maintainability, and scalability.



## **main.c**

Handles program execution and user interaction



## **fileio.c**

Manages loading, saving, and file existence checking



## **timetable.c**

Handles slot checking, updating, and initialization



## **ui.c**

Manages display, input, and batch selection

# Data Structures Used

## BatchSlot Structure

```
typedef struct {  
    char faculty[50];  
} BatchSlot;
```

Stores faculty name assigned to batch slot

## FacultySlot Structure

```
typedef struct {  
    char batch[50];  
    char subject[50];  
} FacultySlot;
```

Stores batch and subject names

## Timetable Dimensions

5 Days ( $N = 5$ )

4 Periods per Day ( $M = 4$ )

Total Slots = 20

```
BatchSlot batch[N][M];  
FacultySlot faculty[N][M];
```

# User Interface

The system displays slot view:

Example Output:

Slot 0 (Day 0, Period 0): FREE  
Slot 1 (Day 0, Period 1): OCCUPIED  
Slot 2 (Day 0, Period 2): BLOCKED

## Legend:

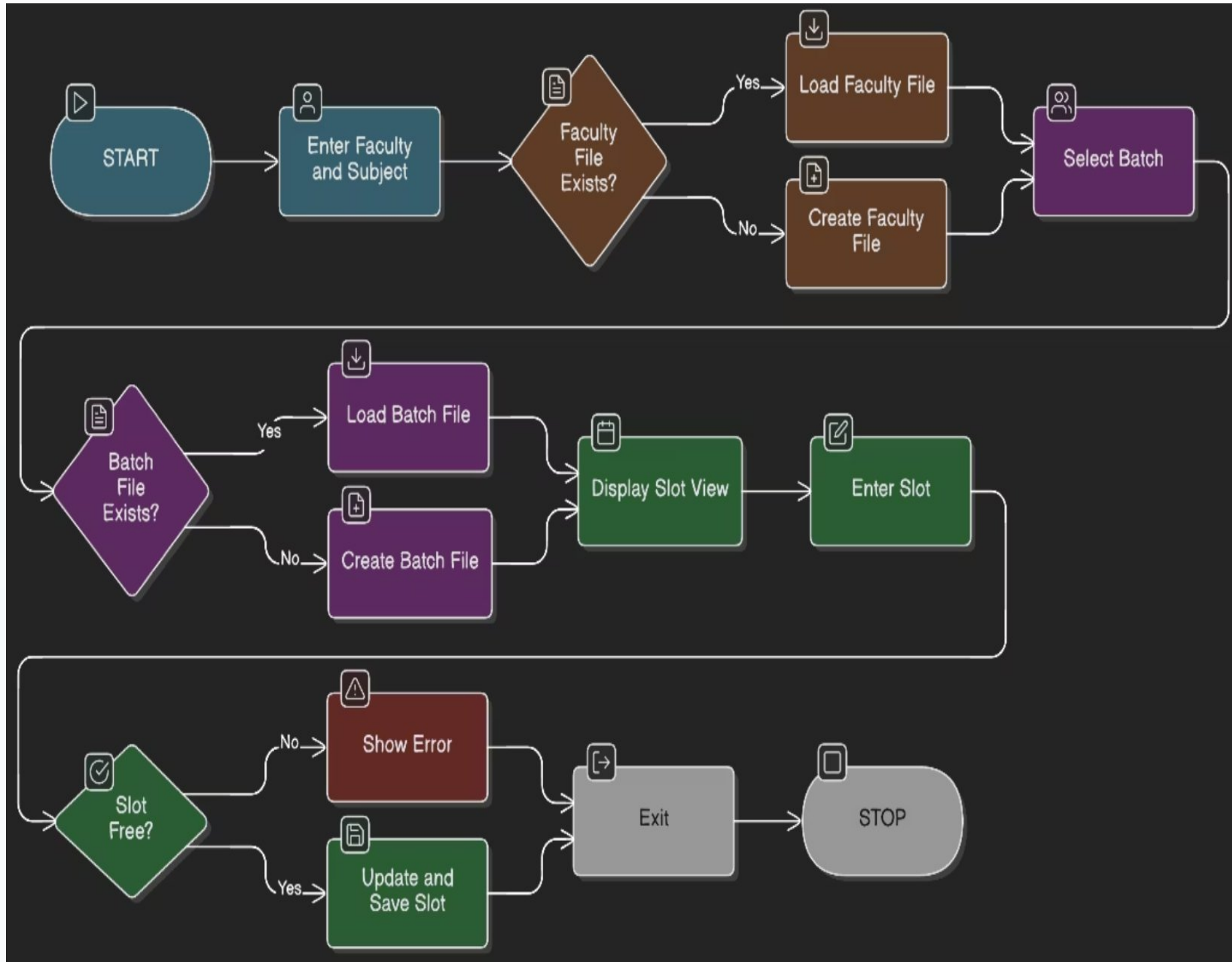
- FREE → Can assign
- OCCUPIED → Batch already taken
- BLOCKED → Faculty already busy

## User can:

- Select slot number
- Exit selection
- Choose different batch



# Workflow & Flowchart



## Workflow

1. Start program
2. Enter faculty name and subject
3. Load or create faculty timetable file
4. Select batch
5. Load or create batch timetable file
6. Display slot status (FREE / OCCUPIED / BLOCKED)
7. Select slot
8. Check if slot is free (both faculty & batch)
9. If free → update and save files
10. Repeat or exit

# File Handling Mechanism

Each timetable is stored in CSV format for persistent storage, easy readability, and external editing.

## Faculty File Example

```
day,period,batch,subject
0,0,EMPTY,EMPTY
1,0,CS101,Programming
1,1,CS202,Database
2,0,CS101,Programming
```

## Batch File Example

```
day,period,faculty
0,0,EMPTY
1,0,Dr. Smith
1,1,Prof. Johnson
2,0,Dr. Smith
```



### file\_exists()

Checks if timetable file exists



### load\_batch()

Loads batch timetable data



### save\_batch()

Saves batch timetable to file



### load\_faculty()

Loads faculty timetable data



### save\_faculty()

Saves faculty timetable to file

# Student Priority Voting System

To make the timetable system more dynamic and student-centric, we propose a **Subject Priority Voting Feature**.

## ◆ Concept

If a subject is falling behind in syllabus completion, students of that batch can vote to prioritize it.

## ◆ How It Works

- Students vote for subjects requiring extra attention
- System records and counts votes
- Subject with highest votes is marked as “High Priority”
- Faculty can allocate additional periods accordingly

## ◆ Benefits

- Ensures syllabus completion on time
- Improves academic planning
- Makes scheduling adaptive
- Encourages student participation
- Reduces academic pressure before exams

## ◆ Impact

Transforms the system from a static timetable generator into a **dynamic, feedback-driven scheduling system**.



# Advantages

## Conflict Prevention

Successfully prevents scheduling conflicts

## Lightweight

Simple and efficient system

## Modular Code

Organized, maintainable structure

## Easy Management

Simple file management

## Scalable

Suitable for small institutions

## Extensible

Easy to modify and enhance



# Conclusion

## **The Faculty Timetable Generator:**

- Successfully prevents scheduling conflicts
- Stores data efficiently using CSV files

## **Demonstrates core C concepts:**

- Structures
- File Handling
- Modular Programming
- 2D Arrays
- String Handling

# THANK YOU